

Komparativna analiza sinhronih komunikacionih paradigmi u IoT mikroservisnim sistemima

Naučno-istraživački i tehnički izveštaj o performansama protokola u edge-cloud kontinuumu

1. Opis i Arhitektura Sistema

Sistem je dizajniran kao kontejnerizovani IoT ekosistem unutar *edge-cloud* kontinuma, implementiran pomoću **Docker Compose-a**. Svrha sistema je skladištenje, praćenje i analitika vremenski serijalizovanih ekoloških podataka (kvalitet vazduha). Za skladištenje se koristi centralizovana **PostgreSQL** baza podataka, optimizovana za IoT podatke kroz kreiranje indeksa nad kolonama mernog vremena i identifikatora uređaja.

Arhitektura se sastoji od tri nezavisna mikroservisa implementirana u dve različite tehnologije (kombinacija .NET okruženja i Node.js-a):

1.1 REST Mikroservis (ASP.NET Core)

- **Tehnologija:** ASP.NET Core / .NET 10 (C#)
- **Komunikacija:** Standardni HTTP/1.1 protokoli, prenos podataka u tekstualnom JSON formatu.
- **Dokumentacija:** Automatski generisana OpenAPI (Swagger) specifikacija.
- **Karakteristike:** Visoka pouzdanost i široka kompatibilnost, ali pati od *over-fetching-a* (uvek vraća kompletan set od 15 dostupnih polja senzora) i tekstualnog overhead-a pri serijalizaciji.

1.2 gRPC Mikroservis (ASP.NET Core)

- **Tehnologija:** .NET 10 (C#)
- **Komunikacija:** HTTP/2 protokol sa trajnim (*persistent*) konekcijama.
- **Format podataka:** Binarni **Protocol Buffers (Protobuf)** definisan preko [airquality.proto](#) ugovorne šeme.
- **Karakteristike:** Izuzetno kompaktan mrežni payload, strogo tipiziran ugovor između klijenta i servera bez prenosa naziva polja u samom paketu.

1.3 GraphQL Mikroservis (Node.js)

- **Tehnologija:** Node.js (JavaScript)
- **Komunikacija:** HTTP/1.1 sa GraphQL izvršnim mehanizmom upita.
- **Karakteristike:** Rešava strukturalni problem *over-fetching-a*, omogućavajući klijentu (npr. mobilnoj aplikaciji sa slabijim protokom) da eksplicitno zahteva samo specifična polja (npr. samo temperaturu i vlažnost vazduha).

2. Rezultati Analize i Evaluacije Performansi

Merenja i simulacije opterećenja sprovedena su kroz tri specifična IoT scenarija pod stabilnim opterećenjem od **500 virtuelnih korisnika (VU)** pomoću alata **k6**.

Scenario A — High-Frequency Ingestion

Simulacija uređaja koji šalje podatke u kratkim intervalima. Fokus je na analizi overhead-a protokola prilikom intenzivnog slanja i upisa mernih podataka putem **POST** zahteva, odnosno **mutation** operacija.

API	Srednja latencija	p95 latencija	RPS (Propusnost)	Greška (%)	Checks	Threshold	Napomena
REST	13.2ms	30.85ms	1475/s	0.00%	100%	✓ p95 < 2s	Stabilan upis, visoko CPU iskorišćenje
gRPC	11.21ms	54.03ms	301/s	~0%*	100%	✓ p95 < 2s	Gubitak konekcija na 500 VU
GraphQL	79.26ms	148.59ms	937/s	0.00%	99.47%	✓ p95 < 2s	Dodatni sloj parsiranja upita

* gRPC ima connection reset greške na 500 VU zbog specifičnosti k6 skripte koja je radila per-request konekcije, ali su sve uspostavljene konekcije uspešno procesirane.

Scenario B — Selective Monitoring

Scenario gde klijent (mobilna aplikacija) sa lošom internet vezom traži samo 2 od 15 dostupnih senzorskih vrednosti (**temperature** i **relativeHumidity**) za poslednjih 10 merenja.

API	Srednja latencija	p95 latencija	RPS (Propusnost)	Greška (%)	Checks	Threshold	Napomena
REST	1.26s	3.5s	125/s	0.00%	81.85%	✗ p95 < 2s	Uvek vraća kompletnu strukturu
gRPC	2.68s	7.9s	62/s	0.00%	100%	✗ p95 < 2s	Vraća kompletan Protobuf objekat
GraphQL	843ms	1.95s	179/s	0.00%	88.45%	✓ p95 < 2s	Vraća isključivo 2 tražena polja

Ključni nalaz za Scenario B: GraphQL je jedini servis koji je uspešno zadovoljio postavljeni vremenski prag jer nativno podržava selekciju polja na strani servera, smanjujući mrežni teret.

Scenario C — Heavy Querying

Složeni analitički upiti nad velikim opsegom istorijskih podataka (agregacija celokupne godine na satnom nivou).

API	Srednja latencija	p95 latencija	RPS (Propusnost)	Greška (%)	Checks	Threshold	Napomena
REST	784ms	2.52s	37/s	0.02%	96.92%	✓ p95 < 5s	SQL date_trunc izvršen na bazi podataka
gRPC	3.54s	12.09s	30/s	0.00%	100%	✗ p95 < 5s	Agregacija rađena u C# memoriji aplikacije
GraphQL	19.65s	60s (Timeout)	8/s	24.90%	71.36%	✗ p95 < 5s	Agregacija rađena u JS memoriji (Node.js)

Ključni nalaz za Scenario C: REST je jedini prošao threshold zahvaljujući pravilnom delegiranju agregacije na SQL nivo unutar baze. gRPC i GraphQL su doživeli degradaciju performansi zbog memorijske agregacije.

3. Analiza Mrežnog Saobraćaja (Payload Size)

Merenjem veličine odgovora za identičan set podataka (čitanje 10 merenja) kroz Postman konzolu, utvrđeni su sledeći parametri mrežnog otiska:

- **REST:** Ukupna veličina paketa iznosi **3 KB** (2.86 KB body). Podaci se šalju sa kompletnom JSON strukturom uključujući nazive svih 15 polja u tekstualnom formatu, što generiše najveći mrežni otisak.
- **GraphQL:** Ukupna veličina paketa iznosi svega **713 bajtova**. S obzirom na to da su zahtevana samo dva polja, struktura JSON odgovora je drastično smanjena, demonstrirajući efikasnost u uštedi mrežnog protoka.
- **gRPC:** Prenos podataka u čistom binarnom formatu uklanja potrebu za slanjem naziva polja (menjaju ih numerički tagovi iz proto datoteke), što pruža maksimalnu optimizaciju prenosnog paketa pri slanju kompletnih objekata.

4. Analiza Potrošnje Hardverskih Resursa

Praćenjem hardverskih resursa preko komande `docker stats` tokom izvršavanja Scenarija A pod punim opterećenjem sa 500 VU, zabeleženi su sledeći pikovi potrošnje:

4.1 Pik opterećenja tokom REST load testiranja

NAME	CPU %	MEM USAGE / LIMIT	MEM %
airquality-grpc	0.00%	1.169GiB / 14.99GiB	7.80%
airquality-rest	275.61%	239.3MiB / 14.99GiB	1.56%
airquality-graphql	0.00%	66.88MiB / 14.99GiB	0.44%
airquality-postgres	130.52%	301.6MiB / 14.99GiB	1.96%

.NET REST servis efikasno paralelizuje zadatke kroz niti i zauzima do 275.61% CPU-a, dok baza podataka angažuje 130.52% procesorske snage za upis stotina hiljada mernih tačaka.

4.2 Pik opterećenja tokom GraphQL load testiranja

NAME	CPU %	MEM USAGE / LIMIT	MEM %
airquality-grpc	0.01%	1.169GiB / 14.99GiB	7.80%
airquality-rest	0.01%	221MiB / 14.99GiB	1.44%
airquality-graphql	90.99%	572.2MiB / 14.99GiB	3.73%
airquality-postgres	54.50%	182.1MiB / 14.99GiB	1.19%

GraphQL servis (Node.js) radi u jedno-nitnom (single-thread) režimu, gde dostiže svoj procesorski limit od 90.99%. Uočljiva je povećana potrošnja RAM memorije (572.2 MiB) usled parsiranja i alokacije kompleksnih objekata stabla upita.

4.3 Pik opterećenja tokom gRPC load testiranja

NAME	CPU %	MEM USAGE / LIMIT	MEM %
airquality-grpc	204.94%	325.5MiB / 14.99GiB	2.12%
airquality-rest	0.01%	221MiB / 14.99GiB	1.44%
airquality-graphql	0.00%	220.6MiB / 14.99GiB	1.44%
airquality-postgres	78.57%	329.9MiB / 14.99GiB	2.15%

gRPC servis dostiže 204.94% CPU iskorišćenja. Zabeleženo je da .NET gRPC kontejner u dugotrajnom radu zadržava alociranu memoriju radi optimizacije mrežnog steka, dok tokom čistog izlovanog testa troši stabilnih 325.5 MiB.

5. Zaključak i Inženjerske Preporuke

Na osnovu sprovedenih k6 simulacija, analiza mrežnih paketa i hardverskih performansi, definisana su tri ključna zaključka za projektovanje IoT sistema:

- **Za Ingestion sloj (Edge u Cloud):** .NET REST mikroservis se pokazao kao najstabilnije rešenje za masovni paralelni upis podataka sa visokom konkurentnošću. Za punu efikasnost gRPC protokola, skripte i klijenti moraju koristiti trajne konekcije (*connection pooling*), umesto stalnog zatvaranja i otvaranja novih soketa.

- **Za klijentski sloj (Cloud u Mobile/Web aplikacije):** GraphQL pruža izuzetne uštede na nivou mrežnog saobraćaja (u ovom testu smanjenje sa 3 KB na 713 bajtova), što ga čini idealnim izborom za klijente sa ograničenim mrežnim resursima.
- **Zlatno pravilo skalabilnosti mikroservisa:** Bez obzira na komunikacioni protokol, masovne agregacije istorijskih podataka se moraju delegirati bazi podataka (SQL nivo). Izvršavanje analitike u aplikativnoj memoriji servera drastično degradira performanse i uzrokuje sistemski krah pri velikom opterećenju.